

**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO,
ENGENHARIA ELETRÔNICA E ENGENHARIA CARTOGRÁFICA**

**RAFAEL DE FIGUEIREDO SCHUINKI
RAQUEL BELCHIOR FAÇANHA NOGUEIRA
MATEUS DA SILVA MALDONADO
ESTÊVÃO JOHNATAS RIBEIRO BATISTA**

**CAIXÃO DE AREIA COM REALIDADE AUMENTADA: SISTEMA AR
SANDBOX PARA MODELAGEM TOPOGRÁFICA INTERATIVA**

**RIO DE JANEIRO
2026**

©2026

INSTITUTO MILITAR DE ENGENHARIA

Praça General Tibúrcio, 80 – Praia Vermelha

Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmар ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

de Figueiredo Schuinki, Rafael; Belchior Façanha Nogueira, Raquel; da Silva Maldonado, Mateus; Johnatas Ribeiro Batista, Estêvão.

Caixão de Areia com Realidade Aumentada: Sistema AR Sandbox para Modelagem Topográfica Interativa / Rafael de Figueiredo Schuinki, Raquel Belchior Façanha Nogueira, Mateus da Silva Maldonado e Estêvão Johnatas Ribeiro Batista. – Rio de Janeiro, 2026.

53 f.

Orientador(es): Daniel Rodrigues dos Santos, Leonardo Assumpção Moreira, Bruno Eduardo Madeira e Gabriel da Cruz Fontenelle.

Projeto Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica, 2026.

1. Realidade Aumentada. 2. Caixão de Areia. 3. Sensor Kinect. 4. Modelo Digital de Elevação. 5. Visão Computacional. 6. Decomposição em Valores Singulares. 7. Projeção Tsai. 8. OpenCV. i. Rodrigues dos Santos, Daniel (orient.) ii. Assumpção Moreira, Leonardo (orient.) iii. Eduardo Madeira, Bruno (orient.) iv. da Cruz Fontenelle, Gabriel (orient.) v. Título

RAFAEL DE FIGUEIREDO SCHUINKI
RAQUEL BELCHIOR FAÇANHA NOGUEIRA
MATEUS DA SILVA MALDONADO
ESTÊVÃO JOHNATAS RIBEIRO BATISTA

CAIXÃO DE AREIA COM REALIDADE AUMENTADA: SISTEMA AR
SANDBOX PARA MODELAGEM TOPOGRÁFICA INTERATIVA

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica.

Orientador(es): Daniel Rodrigues dos Santos, D.Sc
Leonardo Assumpção Moreira, D.Sc
Bruno Eduardo Madeira, D.Sc
Gabriel da Cruz Fontenelle, M.Sc

Rio de Janeiro

2026

**RAFAEL DE FIGUEIREDO SCHUINKI
RAQUEL BELCHIOR FAÇANHA NOGUEIRA
MATEUS DA SILVA MALDONADO
ESTÊVÃO JOHNATAS RIBEIRO BATISTA**

Caixão de Areia com Realidade Aumentada: Sistema AR Sandbox para Modelagem Topográfica Interativa

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia de Computação, Engenharia Eletrônica e Engenharia Cartográfica.

Orientador(es): Daniel Rodrigues dos Santos, Leonardo Assumpção Moreira, Bruno Eduardo Madeira e Gabriel da Cruz Fontenelle.

Aprovada em a ser definida, pela seguinte banca examinadora:

Banca examinadora a ser definida por ocasião da defesa

Rio de Janeiro

2026

Aos nossos familiares, cuja paciência e apoio incondicionais tornaram este trabalho possível.

AGRADECIMENTOS

Agradecemos ao Instituto Militar de Engenharia pela formação de excelência recebida ao longo do curso, e ao nosso orientador pelas discussões sempre enriquecedoras e pela orientação segura durante todo o desenvolvimento deste projeto.

Agradecemos à Biblioteca do IME pelo apoio bibliográfico e às equipes dos laboratórios de Computação e Cartografia pelo suporte técnico.

Agradecemos também aos projetos de código aberto que tornaram este trabalho possível: NumPy, OpenCV, Open3D, rasterio e ao projeto SARndbox da Universidade da Califórnia em Davis.

“The computer was born to solve problems that did not exist before.”
(Bill Gates)

RESUMO

Este trabalho apresenta o desenvolvimento de um **Caixão de Areia com Realidade Aumentada** (*AR Sandbox*), sistema que projeta, em tempo real, uma grade contínua de quadrados coloridos sobre uma caixa de areia física de 1,5 m $\times$ 1,5 m, com o objetivo de guiar o operador na modelagem topográfica de terrenos de acordo com um Modelo Digital de Elevação (MDE) de referência em formato GeoTIFF. Um sensor Microsoft Kinect, montado a 2,5 m de altura, captura a topografia em tempo real; o motor matemático discretiza a superfície em uma malha de 30 $\times$ 30 células (5 cm $\times$ 5 cm) e classifica cada célula por comparação com o MDE: **vermelho** para excesso de areia (cavar), **azul** para areia insuficiente (preencher) e **verde** para dentro da tolerância de 2 cm. A projeção geométrica emprega o modelo de câmera Tsai via `cv2.projectPoints`, com rasterização por polígonos preenchidos (`cv2.fillPoly`). O pipeline matemático abrange: ajuste de plano por Decomposição em Valores Singulares (SVD), construção de base ortonormal via Gram-Schmidt, transformação afim 4 $\times$ 4 e projeção pinhole. O sistema implementa resiliência total por *fallback* em cascata: na ausência do Kinect, opera em modo simulação interativo com pá virtual controlada por mouse; na ausência do GeoTIFF, gera um Morro Gaussiano sintético. A qualidade do motor matemático é assegurada por 26 testes unitários automatizados (*Test-Driven Development*). O sistema foi desenvolvido em Python 3.12 com NumPy e OpenCV como dependências obrigatórias. Além disso, o sensor utilizado é um Kinect Xbox One S adaptado para conexão direta ao PC mediante modificação de hardware, dispensando o adaptador oficial descontinuado pela Microsoft. **A validação realizada neste trabalho restringe-se a testes unitários do motor matemático com valores sintéticos**; a validação de ponta a ponta do sistema com hardware real (captura sobre areia, calibração geométrica na posição definitiva) e com um Modelo Digital de Elevação real do BDGEx permanece como trabalho futuro.

Palavras-chave: Realidade Aumentada. Caixão de Areia. Sensor Kinect. Modelo Digital de Elevação. Visão Computacional. Decomposição em Valores Singulares. Projeção Tsai. OpenCV.

ABSTRACT

This work presents the development of an **Augmented Reality Sandbox** (AR Sandbox), a system that projects, in real time, a continuous grid of colored squares onto a physical $1.5\text{ m} \times 1.5\text{ m}$ sandbox, with the goal of guiding the operator in modeling terrain topography according to a reference Digital Elevation Model (DEM) in GeoTIFF format. A Microsoft Kinect sensor, mounted at 2.5 m, captures the topography in real time; the mathematical engine discretizes the surface into a 30×30 cell mesh ($5\text{ cm} \times 5\text{ cm}$) and classifies each cell by comparison with the DEM: **red** for excess sand (dig), **blue** for insufficient sand (fill), and **green** for within 2 cm tolerance. Geometric projection uses the Tsai camera model via `cv2.projectPoints`, with rasterization by filled polygons (`cv2.fillPoly`). The mathematical pipeline includes: plane fitting via Singular Value Decomposition (SVD), orthonormal basis construction via Gram-Schmidt, 4×4 affine transformation, and pinhole projection. The system implements total resilience through cascading fallback: without a Kinect, it operates in interactive simulation mode with a virtual shovel controlled by mouse; without a GeoTIFF, it generates a synthetic Gaussian hill. Software quality is ensured by 26 automated unit tests (Test-Driven Development). The system was developed in Python 3.12 with NumPy and OpenCV as mandatory dependencies. Furthermore, the sensor used is a Kinect Xbox One S adapted for direct PC connection through hardware modification, bypassing the official adapter discontinued by Microsoft. **The validation carried out in this work is limited to unit tests of the mathematical engine using synthetic values**; end-to-end validation of the system with real hardware (capture over actual sand, geometric calibration at the definitive position) and with a real Digital Elevation Model from BDGEx remains as future work. **Keywords:** Augmented Reality. Sandbox. Kinect

Sensor. Digital Elevation Model. Computer Vision. Singular Value Decomposition. Tsai Projection. OpenCV.

LISTA DE ILUSTRAÇÕES

Figura 1 – Comparação entre o MDE original e o MERIT DEM para o delta do Mekong e a planície amazônica	25
Figura 2 – Pipeline geral da metodologia	28

LISTA DE TABELAS

Tabela 1	– Módulos do sistema AR Sandbox e suas responsabilidades	34
Tabela 2	– Parâmetros de configuração do sistema AR Sandbox	34
Tabela 3	– Padrões de projeto utilizados no sistema AR Sandbox	35
Tabela 4	– Parâmetros do modelo da pá virtual	42
Tabela 5	– Estado visual inicial no modo simulação completo	44
Tabela 6	– Materiais utilizados na adaptação do Kinect Xbox One S	44
Tabela 7	– Cobertura da suíte de testes unitários	47
Tabela 8	– Cronograma previsto para o desenvolvimento do projeto.	52

LISTA DE ABREVIATURAS E SIGLAS

AR	<i>Augmented Reality</i> (Realidade Aumentada)
ASTER	<i>Advanced Spaceborne Thermal Emission and Reflection Radiometer</i>
BDGEx	Banco de Dados Geográficos do Exército
BGR	<i>Blue, Green, Red</i> (convenção de cores do OpenCV)
DEM	<i>Digital Elevation Model</i> (Modelo Digital de Elevação)
GDEM	<i>Global Digital Elevation Model</i>
GeoTIFF	<i>Geographic Tagged Image File Format</i> (formato <i>raster</i> georreferenciado)
GIL	<i>Global Interpreter Lock</i> (trava global do interpretador Python)
IME	Instituto Militar de Engenharia
MDE	Modelo Digital de Elevação
MVC	<i>Model-View-Controller</i> (padrão de arquitetura de software)
OOP	<i>Object-Oriented Programming</i> (Programação Orientada a Objetos)
PFC	Projeto Final de Curso
RGBD	<i>Red, Green, Blue, Depth</i> (imagem com informação de profundidade)
SIRGAS	Sistema de Referência Geocêntrico para as Américas
SRTM	<i>Shuttle Radar Topography Mission</i>
SVD	<i>Singular Value Decomposition</i> (Decomposição em Valores Singulares)
TDD	<i>Test-Driven Development</i> (Desenvolvimento Guiado por Testes)
TIN	<i>Triangulated Irregular Network</i> (Rede Triangular Irregular)
USB	<i>Universal Serial Bus</i>
UTM	<i>Universal Transverse Mercator</i> (sistema de projeção cartográfica)

LISTA DE SÍMBOLOS

$\mathbf{n}$	Vetor normal ao plano da mesa
$\bar{\mathbf{p}}$	Centroide da nuvem de pontos
M	Matriz centralizada dos pontos ($N \times 3$)
U, Σ, V^T	Fatores da Decomposição SVD
σ_i	Valores singulares ($\sigma_1 \geq \sigma_2 \geq \sigma_3$)
$X_{\text{mesa}}, Y_{\text{mesa}}, Z_{\text{mesa}}$	Base ortonormal do referencial da mesa
R	Matriz de rotação (3×3)
T	Matriz de transformação afim (4×4)
$\mathbf{t}$	Vetor de translação
K	Matriz intrínseca do projetor
f_x, f_y	Distâncias focais do projetor (pixels)
c_x, c_y	Ponto principal do projetor (pixels)
$C_{i,j}$	Célula (i, j) da grade de discretização
$\Delta x, \Delta y$	Dimensões da célula da grade (metros)
$\bar{Z}_{i,j}^{\text{real}}$	Altura média real da célula (i, j)
$Z_{i,j}^{\text{MDE}}$	Altura alvo do MDE no centro da célula (i, j)
τ	Tolerância de cor (metros); padrão: $\tau = 0,02$ m
$\mathbf{H}$	Matriz de alturas da areia virtual (simulação)
α	Intensidade da pá virtual (metros/evento)
σ	Desvio padrão do perfil Gaussiano da pá virtual
r	Raio de ação da pá virtual (metros)
L_x, L_y	Dimensões da mesa (1,50 m cada)
N_x, N_y	Número de células da grade por eixo (30 cada)
h_{max}	Altura máxima de areia (0,30 m)

Ω	Domínio espacial da área de estudo (curvas de nível)
$f(x, y)$	Função escalar de relevo (altitude verdadeira)
$\hat{f}(x, y)$	Superfície altimétrica interpolada a partir das curvas
h	Cota altimétrica de uma curva de nível
E	Equidistância altimétrica entre curvas de nível (m)
$\gamma(s)$	Parametrização de uma curva de nível pelo comprimento s
Δs	Espaçamento de amostragem ao longo da curva (m)
r_{MDE}	Resolução espacial da grade <i>raster</i> do MDE (m)
$\lambda_1, \lambda_2, \lambda_3$	Coordenadas baricêntricas (interpolação no triângulo)
M_{MDE}	Matriz altimétrica do MDE <i>raster</i> gerado
Z_{norm}	Altitude normalizada ($\in [0, 1]$)
Z_{mesa}	Altura alvo na escala física do caixão de areia (m)
φ, λ	Latitude e longitude (coordenadas geográficas)

SUMÁRIO

1	INTRODUÇÃO	17
1.1	Contextualização: Modelos Digitais de Elevação	17
1.2	Motivação	18
1.3	Objetivos	19
1.3.1	Objetivo Geral	19
1.3.2	Objetivos Específicos	19
1.4	Escopo e Limitações do Trabalho	19
1.5	Estrutura do Documento	20
2	REFERENCIAL TEÓRICO	22
2.1	Sensores de Profundidade	22
2.1.1	Microsoft Kinect v1	22
2.1.2	Microsoft Kinect v2	22
2.1.3	Azure Kinect e RealSense	23
2.2	Modelos de Câmera e Calibração	23
2.2.1	Modelo Pinhole	23
2.2.2	Modelo de Tsai	23
2.2.3	Decomposição em Valores Singulares	23
2.3	Modelos Digitais de Elevação	24
2.3.1	MERIT DEM: Correção de Erros em MDEs Globais	24
3	TRABALHOS RELACIONADOS	26
3.1	AR Sandboxes — Estado da Arte	26
3.1.1	Variantes e Adoção Acadêmica	26
3.1.2	Diversificação de Hardware	26
3.1.3	Posicionamento do Presente Trabalho	27
4	METODOLOGIA	28
4.1	Aquisição de Insumos (MDE)	28
4.1.1	Aquisição através do BDGEx	29
4.1.2	Aquisição de Dados Vetoriais a partir do BDGEx	29
4.1.2.1	Curvas de Nível como Isolinhas Altimétricas	29
4.1.2.2	Leitura, Reprojeção e Pré-processamento	30
4.1.2.3	Amostragem em Nuvem de Pontos Tridimensionais	31
4.1.2.4	Triangulação de Delaunay e Interpolação Baricêntrica	31
4.1.2.5	Discretização em Grade, Exportação GeoTIFF e Normalização	32

4.1.3	Aquisição a partir de Imagens SRTM, ASTER	32
4.2	Software de Projeção do MDE	33
4.2.1	Visão Geral — Arquitetura Monolítica Modularizada	33
4.2.2	Parâmetros Físicos do Sistema	34
4.2.3	Máquina de Estados	34
4.2.4	Padrões de Projeto	35
4.2.5	Saídas Visuais	35
4.3	Motor Matemático — Pipeline Completo	35
4.3.1	Passo 1 — Ajuste de Plano por SVD	36
4.3.2	Passo 2 — Base Ortonormal da Mesa (Gram-Schmidt)	36
4.3.3	Passo 3 — Transformação Afim 4×4 (Kinect → Mesa)	37
4.3.4	Passo 4 — Projeção Pinhole/Tsai ($3D \rightarrow 2D$)	38
4.4	Renderização por Malha Discretizada	39
4.4.1	Definição da Grade	39
4.4.2	Agregação Espacial — Média de Altura por Célula	39
4.4.3	Regra de Negócio — Coloração por Diferença de Altitude	40
4.4.4	Projeção em Lote e Rasterização	41
4.5	Emulador Interativo de Areia	41
4.5.1	Estado Persistente da Areia	42
4.5.2	Modelo Matemático da Pá Virtual	42
4.5.3	Mapeamento Pixel → Coordenada Física	42
4.6	Mecanismo de Resiliência — Fallback em Cascata	43
4.6.1	Sensor — KinectSensor	43
4.6.2	MDE — AdaptadorMDE	43
4.6.3	Estado Visual Inicial da Simulação	44
4.7	Adaptação e Integração do Sensor Kinect Xbox One S	44
4.7.1	Adaptação de Hardware	44
4.7.2	Compatibilidade de Software — pykinect2 e Python 3.12	45
4.7.3	Script de Diagnóstico e Validação	45
5	RESULTADOS	46
5.1	Estratégia TDD	46
5.2	Cobertura por Componente	46
5.3	Exemplos de Transparência Matemática	46
6	CONCLUSÃO	48
6.1	Conclusões	48
6.2	Trabalhos Futuros	48
	REFERÊNCIAS	50

APÊNDICE A – CRONOGRAMA DE EXECUÇÃO DO PROJETO	52
APÊNDICE B – CORREÇÕES DA BIBLIOTECA PYKINECT2 PARA PYTHON 3.12	53

1 INTRODUÇÃO

O interesse em sistemas de visualização geoespacial interativos cresceu substancialmente na última década, impulsionado pela convergência entre sensores de profundidade de baixo custo, hardware gráfico acessível e bibliotecas de visão computacional maduras. Nesse contexto, o **Caixão de Areia com Realidade Aumentada** (*AR Sandbox*) emerge como uma plataforma de alto valor pedagógico e operacional: ao projetar informações topográficas diretamente sobre uma superfície de areia física, o sistema cria um ciclo de retroalimentação tangível entre o modelo digital e a manipulação manual do terreno (KREYLOS; CAVES, 2013; KREYLOS, 2016).

Este Projeto Final de Curso, desenvolvido no âmbito conjunto das Engenharias de Computação, Eletrônica e Cartográfica do Instituto Militar de Engenharia, apresenta a concepção, implementação e validação de um sistema AR Sandbox de uso militar e educacional. O sistema projeta, em tempo real, uma grade contínua de quadrados coloridos sobre uma caixa de areia física de 1,5 m $\times$ 1,5 m, orientando o operador a modelar o terreno até que a topografia replique um Modelo Digital de Elevação (MDE) de referência.

1.1 Contextualização: Modelos Digitais de Elevação

A representação da superfície do terreno é uma preocupação comum a engenheiros civis, geólogos, geomorfólogos e cartógrafos, que, apesar de interesses distintos, compartilham o desejo de reproduzir o relevo de forma conveniente e com determinada acurácia (LI; ZHU; GOLD, 2005). Historicamente, essa representação evoluiu de pinturas e mapas semi-simbólicos, de baixa qualidade métrica, para os mapas topográficos modernos, que empregam um sistema de símbolos bem definido e uma base matemática consistente, representando a altitude e a morfologia do terreno por meio de curvas de nível (LI; ZHU; GOLD, 2005).

Com o avanço da computação, da matemática moderna e da computação gráfica a partir de meados do século XX, surgiram as técnicas de representação digital do terreno, consolidadas no conceito de **Modelo Digital do Terreno** (MDT). Introduzido por Miller e Laflamme em 1958, o MDT é definido como a representação estatística da superfície contínua do solo por um grande número de pontos selecionados com coordenadas X , Y e Z conhecidas (LI; ZHU; GOLD, 2005). Formalmente, um MDT é um conjunto ordenado de pontos amostrados que representam a distribuição espacial de um ou mais tipos de informação do terreno, sendo comumente tratado como uma representação 2,5-D do espaço geográfico tridimensional (LI; ZHU; GOLD, 2005). Quando a única informação considerada é a altitude, o modelo reduz-se ao **Modelo Digital de Elevação** (MDE), que constitui,

portanto, o subconjunto mais fundamental do MDT (LI; ZHU; GOLD, 2005).

A importância dos MDEs decorre de vantagens que a representação analógica tradicional não oferece: a variedade de formas de representação derivadas (mapas topográficos, perfis, seções e animações 3-D), a ausência de perda de acurácia ao longo do tempo, a maior viabilidade de automação e processamento em tempo real, e a facilidade de representação em múltiplas escalas (LI; ZHU; GOLD, 2005). Tais características tornaram os MDEs elementos centrais das infraestruturas nacionais de dados geoespaciais, com aplicação em planejamento e projeto de obras de engenharia, análise de bacias hidrográficas, análise geomorfológica, estudos de visibilidade, cálculo de volumes e, de forma expressiva, em aplicações militares como simulação de voo, simulação de combate, planejamento de rotas e navegação (LI; ZHU; GOLD, 2005).

É justamente nesse último domínio que se insere o **caixão de areia** (*sandbox*). Modelos físicos de terreno sempre atraíram militares, planejadores e engenheiros civis; originalmente confeccionados em borracha, plástico, argila e areia, foram amplamente empregados no planejamento de operações militares — como registrado durante a Segunda Guerra Mundial e na Guerra das Malvinas (1982), quando as forças britânicas em campo usaram extensivamente maquetes de areia e argila (LI; ZHU; GOLD, 2005). O caixão de areia condensa, assim, a virtude pedagógica e operacional de tornar o relevo tangível e manipulável. O presente trabalho propõe unir essa tradição centenária à precisão dos MDEs digitais: ao projetar sobre a areia física, em tempo real, a comparação entre a topografia modelada e um MDE de referência, o sistema AR Sandbox recupera o valor intuitivo da maquete física e o potencializa com a acurácia, a atualização e a automação características da modelagem digital do terreno.

1.2 Motivação

A modelagem de terrenos em maquetes físicas é uma prática centenária no treinamento militar, porém limitada pela ausência de retroalimentação em tempo real: o instrutor precisa verificar manualmente se o terreno modelado corresponde ao gabarito. O AR Sandbox elimina essa limitação ao automatizar a comparação topográfica e projetar o resultado diretamente na superfície.

Do ponto de vista cartográfico, o sistema serve como ferramenta didática para a interpretação de curvas de nível e MDEs, tornando conceitos abstratos concretos e manipuláveis. Para a engenharia de computação, o projeto integra algoritmos de álgebra linear, visão computacional e processamento de sinais em um pipeline concebido para operar em tempo real, com requisitos de latência e robustez que orientaram as escolhas de arquitetura descritas no Capítulo 4, ainda que sua mensuração quantitativa não tenha sido realizada neste trabalho (Seção 1.4).

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver um sistema AR Sandbox capaz de capturar a topografia de uma caixa de areia por meio de um sensor de profundidade, compará-la com um MDE de referência e projetar o resultado da comparação diretamente na superfície da areia como uma grade colorida de tempo real.

1.3.2 Objetivos Específicos

1. Implementar um pipeline de calibração geométrica baseado em SVD e Gram-Schmidt para estabelecer o referencial da mesa.
2. Desenvolver um mecanismo de discretização em malha para agregação espacial e filtragem do ruído do sensor Kinect.
3. Integrar o modelo de projeção pinhole/Tsai para o mapeamento geométrico correto entre o espaço 3D e o plano de projeção 2D.
4. Implementar um adaptador para Modelos Digitais de Elevação em formato GeoTIFF com interpolação bilinear.
5. Garantir resiliência operacional por meio de mecanismo de *fallback* em cascata para operação sem hardware físico.
6. Validar o motor matemático com uma suíte de testes unitários automatizados cobrindo todos os componentes críticos.
7. Adaptar o sensor Kinect Xbox One S para conexão direta ao PC mediante modificação de hardware, garantindo compatibilidade com a biblioteca `pykinect2` no Python 3.12.

1.4 Escopo e Limitações do Trabalho

É importante delimitar, já na Introdução, o escopo de validação efetivamente alcançado por este trabalho, para que os resultados apresentados no Capítulo 5 sejam interpretados corretamente. O desenvolvimento foi conduzido e verificado majoritariamente em bancada e por meio de testes unitários automatizados com valores sintéticos e *hardcoded*; a validação de ponta a ponta do sistema completo, sobre hardware físico e com dados cartográficos reais, permanece como trabalho futuro (Seção 6.2). Especificamente:

- o sensor Kinect adaptado (Seção 4.7) foi testado, em bancada, apenas quanto ao recebimento correto de *frames* de profundidade, e não sobre a superfície de areia real na configuração definitiva (mesa de 1,5 m × 1,5 m, sensor a 2,5 m de altura);

- a calibração geométrica por SVD e Gram-Schmidt (Seções 4.3.1 e 4.3.2) foi verificada apenas com dados sintéticos nos testes unitários (Capítulo 5), e ainda não foi executada com dados reais capturados na posição definitiva do sensor;
- o pipeline cartográfico completo – curvas de nível $\rightarrow$ TIN $\rightarrow$ GeoTIFF $\rightarrow$ normalização (Seção 4.1.2) – foi implementado e testado com dados de exemplo, mas ainda não foi executado com um MDE real obtido do BDGEx;
- a via alternativa de aquisição a partir de modelos globais (SRTM, ASTER GDEM; Seção 4.1.3) é conceitualmente compatível com a arquitetura do sistema, mas não foi exercitada neste trabalho;
- a calibração extrínseca mesa $\rightarrow$ projektor não está documentada no texto atual (Seção 4.3.4);
- não foram realizadas medições de desempenho em tempo real (taxa de quadros, latência) nem de acurácia de reprojeção geométrica, apesar de o pipeline ter sido concebido com tais requisitos em mente (Seção 4.3);
- a caracterização de ruído do sensor (Seção 4.4.3) e a interpolação TIN a partir de curvas de nível (Seção 4.1.2.4) recorrem a aproximações da literatura (ruído do Kinect v1 como proxy do v2; ausência de tratamento do efeito *terraço*), cuja adequação ao caso real ainda não foi verificada;
- o documento ainda não inclui fotografias da adaptação de hardware, da mesa física ou capturas de tela das saídas visuais do sistema; essas evidências fotográficas permanecem pendentes para a versão final.

Essas pendências estão detalhadas como trabalhos futuros no Capítulo 6, e sua leitura é recomendada em conjunto com o Capítulo 5, cujos resultados se restringem à correção matemática do motor de cálculo, verificada por testes unitários com dados sintéticos.

1.5 Estrutura do Documento

O presente documento está organizado da seguinte forma:

- **Capítulo 2:** Referencial teórico sobre sensores de profundidade, modelos de câmera e o formato dos Modelos Digitais de Elevação.
- **Capítulo 3:** Trabalhos relacionados e estado da arte em AR Sandboxes.

- **Capítulo 4:** Metodologia adotada, incluindo arquitetura de software, pipeline matemático completo (SVD, Gram-Schmidt, transformação afim e projeção Tsai), renderização por malha discretizada, emulador interativo, mecanismo de resiliência e adaptação do sensor Kinect Xbox One S para integração com o sistema.
- **Capítulo 5:** Resultados obtidos, com ênfase em qualidade de software e cobertura por testes unitários (TDD).
- **Capítulo 6:** Conclusões e trabalhos futuros.

2 REFERENCIAL TEÓRICO

2.1 Sensores de Profundidade

2.1.1 Microsoft Kinect v1

O Kinect v1, lançado em 2010, utiliza luz estruturada para estimar profundidade com resolução de 640×480 pixels e faixa de operação de 0,5 a 4,5 m, apresentando ruído Gaussiano de $\sigma \approx 3$ mm na faixa de 2 a 3 m (KHOSHELHAM; ELBERINK, 2012). Para o acesso programático ao dispositivo, a biblioteca `libfreenect` (OpenKinect) fornece interface de baixo nível (OpenKinect Community, 2023).

2.1.2 Microsoft Kinect v2

O Kinect v2, lançado em 2013 como acessório do Xbox One, representa uma evolução significativa em relação ao seu predecessor ao adotar sensoramento por *Time-of-Flight* (ToF), o que lhe confere resolução de profundidade de 512×424 pixels e faixa de operação de 0,5 a 4,5 m. Diferentemente do Kinect v1, a comunicação com o PC é realizada exclusivamente via USB 3.0, sendo essa versão do barramento obrigatória para garantir a largura de banda necessária à transmissão dos *frames* de profundidade. O acesso programático ao dispositivo é fornecido pelo *Kinect for Windows SDK 2.0* (Microsoft) em conjunto com a biblioteca `pykinect2`, sendo este o sensor adotado no presente trabalho.

Ruído do Kinect v2. O valor $\sigma \approx 3$ mm citado para o Kinect v1 (Seção anterior) não se aplica diretamente ao v2: por operar por *Time-of-Flight* em vez de luz estruturada, o v2 apresenta comportamento de ruído distinto e, em geral, inverso ao do v1 em função da distância – enquanto a precisão do v1 degrada com o aumento do alcance, a acurácia média do v2 permanece relativamente estável ao longo da faixa operacional, mas o desvio-padrão (ruído por pixel) tende a **crescer** com a distância de operação, sendo tipicamente da ordem de poucos milímetros a curta distância ($\leq 1,5$ m) e podendo superar essa faixa nas proximidades do limite superior de alcance (4,5 m) (WASENMÜLLER; STRICKER, 2017). O Kinect v2 também está sujeito a efeitos de *flying pixels* e interferência multipercurso nas bordas de objetos, ausentes no v1 (WASENMÜLLER; STRICKER, 2017). Não foi localizada, na literatura consultada, uma caracterização específica do ruído do v2 na distância de trabalho deste projeto (2,5 m); a ausência dessa medição é tratada como limitação do escopo (Seção 1.4).

2.1.3 Azure Kinect e RealSense

Sensores mais recentes, como o Azure Kinect (Microsoft) e o RealSense (Intel), oferecem maior resolução e menor ruído em comparação às gerações anteriores. Além disso, ambos são suportados pelo Open3D (ZHOU; PARK; KOLTUN, 2018), biblioteca de geometria 3D *open-source* que provê API unificada para diversas câmeras de profundidade, simplificando a integração em sistemas como o desenvolvido neste trabalho.

2.2 Modelos de Câmera e Calibração

2.2.1 Modelo Pinhole

O modelo de câmera *pinhole* descreve a projeção perspectiva de um ponto 3D (X, Y, Z) em coordenadas de câmera para o pixel (u, v) na imagem:

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y \quad (2.1)$$

onde f_x, f_y são as distâncias focais e (c_x, c_y) é o ponto principal (HARTLEY; ZISSERMAN, 2003).

2.2.2 Modelo de Tsai

Tsai (TSAI, 1987) propôs uma técnica versátil de calibração de câmera que estende o modelo *pinhole* com coeficientes de distorção radial (k_1, k_2, k_3) e tangencial (p_1, p_2) . O OpenCV implementa o modelo de Tsai em `cv2.projectPoints` e `cv2.calibrateCamera` (OpenCV Team, 2024).

2.2.3 Decomposição em Valores Singulares

A SVD de uma matriz $M \in \mathbb{R}^{m \times n}$ produz a fatoração:

$$M = U \Sigma V^T \quad (2.2)$$

onde $U \in \mathbb{R}^{m \times m}$ e $V \in \mathbb{R}^{n \times n}$ são matrizes ortogonais e Σ é diagonal com valores singulares não-negativos em ordem decrescente. O vetor correspondente ao menor valor singular de V^T é a solução de mínimos quadráticos de $\min_{\|\mathbf{v}\|=1} \|M\mathbf{v}\|$ (GOLUB; LOAN, 2013), propriedade explorada no ajuste de plano.

2.3 Modelos Digitais de Elevação

Um MDE é uma representação *raster* da variação altimétrica da superfície terrestre. O formato GeoTIFF encapsula dados *raster* georreferenciados junto com metadados de projeção cartográfica e transformação afim entre pixels e coordenadas geográficas. A biblioteca rasterio (GILLIES et al., 2024) fornece leitura de alta performance de GeoTIFFs em Python. Os fundamentos matemáticos e o procedimento de obtenção do MDE de referência utilizado neste trabalho — por via *raster* direta do BDGEx, por derivação de curvas de nível vetoriais ou por fontes globais como SRTM e ASTER GDEM — são apresentados na abertura do Capítulo 4 (Seção 4.1), por se tratar de conteúdo aplicado diretamente à metodologia do sistema.

2.3.1 MERIT DEM: Correção de Erros em MDEs Globais

Um problema recorrente em MDEs de cobertura global obtidos por sensoriamento remoto (Seção 4.1.3) é a presença de erros sistemáticos e aleatórios que degradam a representação do relevo, sobretudo em áreas de baixa declividade. O MERIT DEM (*Multi-Error-Removed Improved-Terrain DEM*) é um produto derivado que corrige quatro componentes de erro identificados em MDEs espaciais de referência (SRTM3, AW3D e Viewfinder Panoramas): viés absoluto, ruído em faixas (*stripe noise*), ruído do tipo *speckle* e viés de altura de dossel florestal (YAMAZAKI et al., 2017). Após essa correção, a proporção de áreas terrestres mapeadas com acurácia vertical melhor que 2 m aumentou de 39% para 58% globalmente, com ganhos particularmente expressivos em grandes planícies de inundação e florestas alagadas (YAMAZAKI et al., 2017).

A Figura 1 ilustra esse efeito no delta do rio Mekong e na planície de inundação amazônica: o MDE original (linha inferior de cada par) apresenta ruído em faixas e distorções locais visíveis, substancialmente atenuados no MERIT DEM (linha superior), o que revela com mais nitidez a rede de drenagem e a morfologia do terreno.

Esse exemplo reforça, de forma independente, a discussão da Seção 4.1.2.4: MDEs de origem remota estão sujeitos a artefatos de naturezas distintas (ruído de sensor, efeito terraço, viés sistemático), cuja caracterização e correção — quando aplicáveis — afetam diretamente a qualidade da superfície de referência utilizada por sistemas como o deste trabalho.

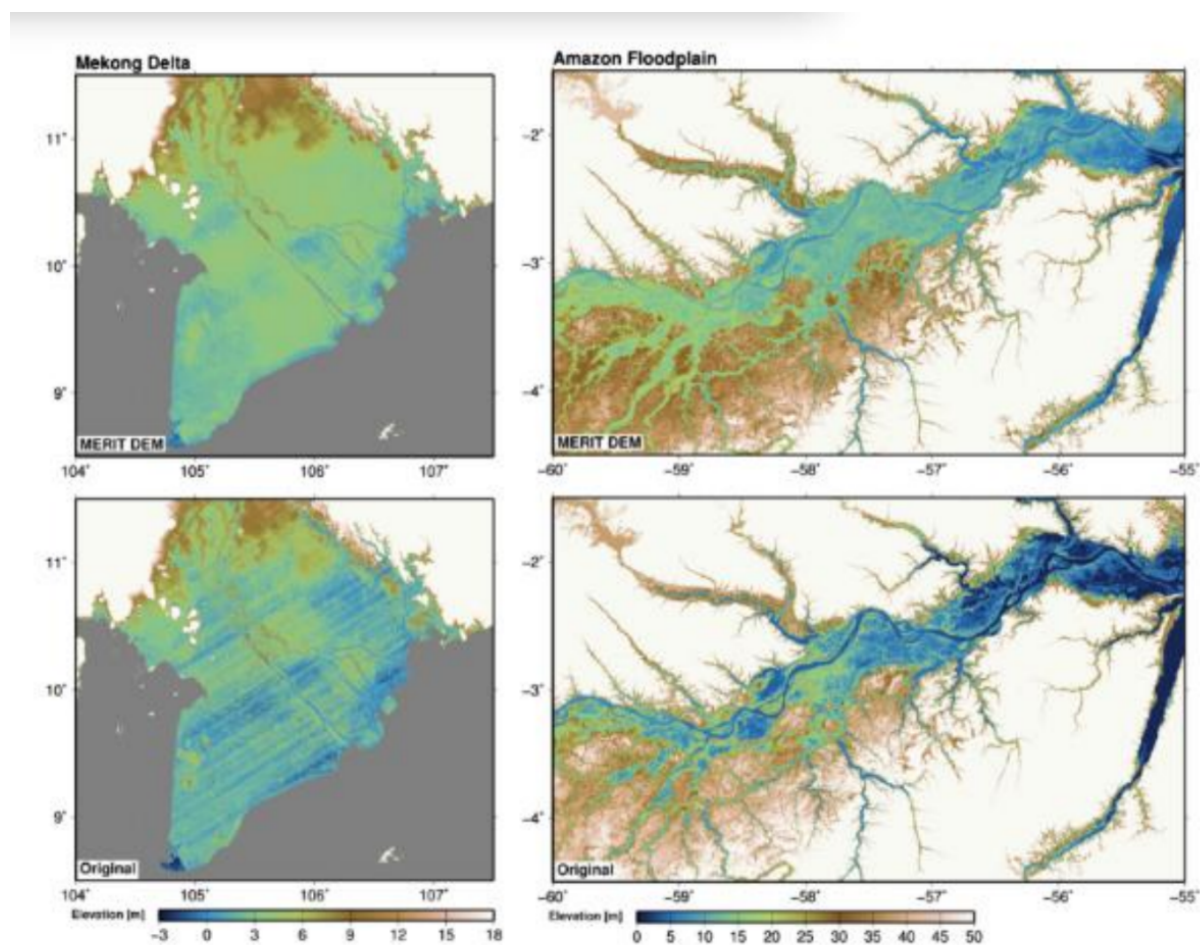


Figura 1 – Comparação entre o MDE original e o MERIT DEM para o delta do Mekong e a planície amazônica

Fonte: (YAMAZAKI et al., 2017).

3 TRABALHOS RELACIONADOS

3.1 AR Sandboxes — Estado da Arte

O conceito de caixa de areia aumentada foi popularizado pelo projeto *SARndbox* da Universidade da Califórnia em Davis (KREYLOS; CAVES, 2013; KREYLOS, 2016). O *SARndbox* projeta mapas de altura e simulações hidrológicas em tempo real sobre uma caixa de areia, usando um sensor Kinect para captura e um projetor calibrado para a saída visual. O *SARndbox* foi, por sua vez, inspirado em um protótipo anterior — *Sandy Station*, demonstrado por pesquisadores tchecos em 2011 — e, desde sua publicação em 2013, deu origem a mais de cem instalações distintas ao redor do mundo (WOODS et al., 2016).

3.1.1 Variantes e Adoção Acadêmica

Diversos grupos independentes replicaram e adaptaram a arquitetura original do *SARndbox* para contextos educacionais específicos, mantendo o mesmo par sensor-projetor mas alterando o conteúdo pedagógico projetado. O projeto *Shaping Watersheds*, da parceria entre UC Davis, Lawrence Hall of Science e o ECHO Lake Aquarium, estendeu o software original com controles adicionais de precipitação e batimetria para o ensino de hidrologia de bacias (REED et al., 2014). Na Universidade Estadual do Leste da Carolina do Norte (ECU), Woods et al. (WOODS et al., 2016) conduziram um estudo piloto de dois semestres integrando o AR Sandbox a laboratórios introdutórios de geologia, reportando ganhos percebidos de aprendizagem na leitura de mapas topográficos e na visualização de processos fluviais e costeiros — trabalho relevante por avaliar sistematicamente o impacto pedagógico do dispositivo, aspecto ausente na literatura original do *SARndbox*.

3.1.2 Diversificação de Hardware

Embora a maioria das implementações reportadas na literatura utilize o Kinect (v1 ou v2) como sensor de captura, a descontinuação desses dispositivos pela Microsoft tem motivado a adoção de sensores alternativos em projetos derivados, como o Intel RealSense e o Asus Xtion Pro, conforme documentado por implementações acadêmicas independentes do AR Sandbox (LÖHR, 2019). Mais recentemente, o Azure Kinect — sucessor direto do Kinect v2 — tem sido avaliado como substituto nessas arquiteturas, apresentando repetibilidade superior à das gerações anteriores (TÖLGYESSY et al., 2021), o que o torna candidato natural para futuras revisões de projetos como o aqui apresentado, inclusive por já ser suportado pelo Open3D (Seção 4.2).

3.1.3 Posicionamento do Presente Trabalho

O presente trabalho difere das variantes acima em dois aspectos fundamentais: (1) a saída visual é uma grade discreta de quadrados coloridos orientada à comparação binária com um gabarito (cavar/preencher/OK) para treinamento militar, e não curvas de nível contínuas ou simulação hidrológica voltada à educação em geociências; (2) o sistema implementa um modo de simulação interativo completo que dispensa hardware físico, garantindo demonstrabilidade plena mesmo na ausência da caixa de areia, do sensor Kinect ou do projetor — mecanismo não relatado nas variantes educacionais citadas. Ressalva-se, no entanto, que a comparação acima é conduzida no nível conceitual e bibliográfico: como discutido no Capítulo 5 e na Seção 1.4, o presente trabalho ainda carece da validação end-to-end com hardware real necessária para confirmar empiricamente essas diferenças frente aos sistemas citados.

4 METODOLOGIA

Este capítulo descreve a metodologia empregada no desenvolvimento do sistema, estruturada nas quatro etapas ilustradas na Figura 2: aquisição dos insumos cartográficos (Seção 4.1), software de projeção do MDE (Seção 4.2), motor matemático (Seção 4.3) e renderização por malha discretizada (Seção 4.4).

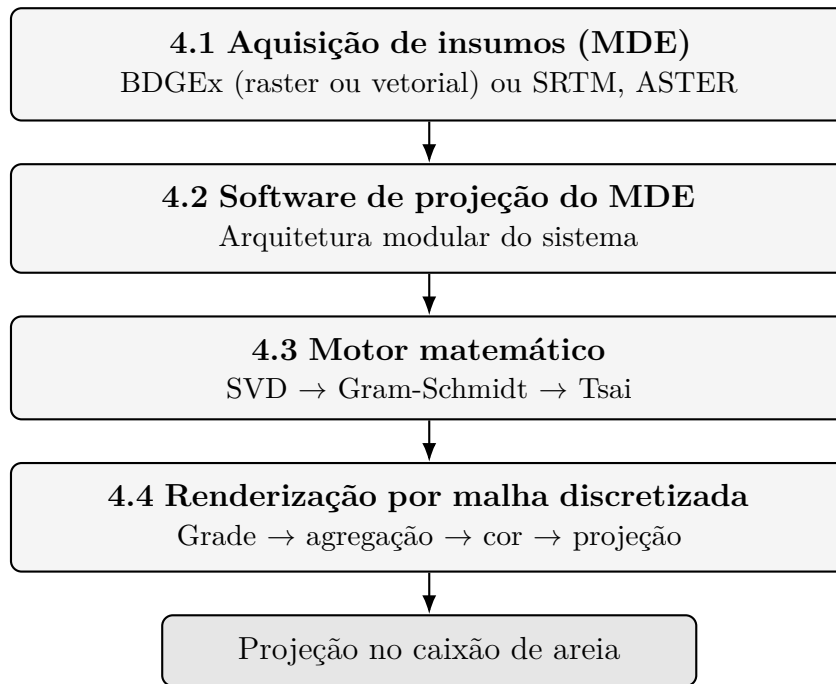


Figura 2 – Pipeline geral da metodologia

Fonte: Elaborado pelos autores.

4.1 Aquisição de Insumos (MDE)

O sistema proposto utiliza um Modelo Digital de Elevação (MDE) como superfície de referência para orientar a modelagem física do terreno no caixão de areia: o motor matemático compara, célula a célula, o relevo medido pelo Kinect sobre a areia com a altura-alvo $Z_{\text{mesa}}(x, y)$ derivada desse MDE (Seções 4.4 e 4.6). Esta seção descreve as três vias consideradas para a obtenção desse insumo, apresentadas em ordem de preferência conforme a disponibilidade de dados para a área de interesse: (i) obtenção direta de um produto *raster* do BDGEx; (ii) derivação a partir de dados vetoriais (curvas de nível) do BDGEx, quando o produto *raster* não está disponível na resolução necessária; e (iii) obtenção de modelos digitais de elevação de cobertura global (SRTM, ASTER GDEM), quando a área de interesse está fora da cobertura do BDGEx ou o acesso a essa base não está disponível. Qualquer que seja a via, o resultado converge para um GeoTIFF

de cotas reais que, normalizado conforme a Seção 4.1.2.5, alimenta o `AdaptadorMDE` (`mde_cartografia.py`).

4.1.1 Aquisição através do BDGEx

O Banco de Dados Geográficos do Exército (BDGEx) é a fonte primária de dados cartográficos considerada neste trabalho. Mediante acesso autorizado nível 5, a base pode disponibilizar tanto produtos vetoriais (curvas de nível em Shapefile, sempre presentes para a área de estudo e detalhadas na Seção 4.1.2) quanto, para algumas áreas, produtos *raster* já discretizados (Modelos Digitais de Terreno/Elevação em GeoTIFF ou formato conversível). Quando um produto *raster* está disponível para a área de interesse, essa é a via de aquisição preferencial: o arquivo é lido diretamente pelo `AdaptadorMDE` com a biblioteca rasterio (GILLIES et al., 2024), dispensando a derivação a partir de curvas de nível descrita na Seção 4.1.2. Resta apenas aplicar a normalização para a escala física da mesa, comum às três vias de aquisição (Seção 4.1.2.5). Na prática, contudo, a cobertura desses produtos *raster* nem sempre está disponível para toda área de interesse, tornando a derivação a partir de dados vetoriais (Seção 4.1.2) o caminho mais frequentemente adotado neste trabalho.

4.1.2 Aquisição de Dados Vetoriais a partir do BDGEx

Quando o BDGEx não disponibiliza um produto *raster* adequado para a área de interesse, o MDE de referência precisa ser derivado a partir de curvas de nível vetoriais. A cadeia de processamento adotada é sintetizada por

$$\text{BDGEx (Shapefile)} \rightarrow P \rightarrow \text{TIN} \rightarrow \text{GeoTIFF} \rightarrow Z_{\text{mesa}} \rightarrow \text{AdaptadorMDE}, \quad (4.1)$$

detalhada nas cinco subseções a seguir.

4.1.2.1 Curvas de Nível como Isolinhas Altimétricas

A entrada do pipeline é um arquivo Shapefile contendo feições lineares correspondentes a curvas de nível, obtido no BDGEx mediante o acesso descrito na Seção 4.1.1. Cada feição é acompanhada, em sua tabela de atributos, da cota altimétrica correspondente. Na aquisição são registrados os elementos necessários ao processamento posterior: área de interesse, sistema de referência de coordenadas, escala/resolução da base, equidistância altimétrica das curvas (quando disponível) e identificação do campo de atributo que armazena a altitude. Todo o processamento subsequente é realizado localmente, em rotinas Python, sem exposição de informações sensíveis da base original.

As curvas de nível constituem uma forma clássica de descrição cartográfica do relevo: cada curva conecta pontos de mesma altitude, permitindo representar a variação

vertical do terreno em uma superfície plana. Considerando o relevo como uma função escalar

$$z = f(x, y), \quad (4.2)$$

em que (x, y) são coordenadas planimétricas e z é a altitude, uma curva de nível de cota h é definida como

$$C = \{(x, y) \in \Omega \mid f(x, y) = h\}, \quad (4.3)$$

onde Ω representa o domínio espacial da área de estudo. Cada curva C é, portanto, uma isolinha altimétrica. No formato Shapefile, cada curva é armazenada como geometria linear acompanhada por esse atributo de cota: para uma curva C associada à altitude h , vale $z(x, y) = h$ para todo $(x, y) \in C$.

Essa definição evidencia uma propriedade fundamental: as curvas de nível, isoladamente, não constituem uma superfície contínua — fornecem valores conhecidos de altitude apenas ao longo de linhas. Entre duas curvas a altitude precisa ser **estimada**. A geração de um MDE a partir de curvas de nível consiste, então, em reconstruir uma função aproximada $\hat{f}(x, y)$ capaz de atribuir altitudes aos pontos intermediários do domínio, pelo procedimento descrito nas subseções seguintes.

4.1.2.2 Leitura, Reprojeção e Pré-processamento

O Shapefile é lido pela biblioteca GeoPandas, permitindo acesso simultâneo às geometrias e atributos. A inspeção inicial verifica quatro elementos: sistema de referência de coordenadas, tipo geométrico das feições, existência do campo de altitude e presença de valores nulos ou inconsistentes. Caso o arquivo esteja em coordenadas geográficas, é realizada a reprojeção para um sistema projetado em metros — preferencialmente UTM/SIRGAS 2000 no fuso correspondente à área de estudo:

$$T : (\varphi, \lambda) \rightarrow (x, y), \quad (4.4)$$

em que φ é a latitude, λ é a longitude e (x, y) são coordenadas planas em metros. A reprojeção é indispensável porque as etapas seguintes operam sobre medidas lineares: comprimento das curvas, espaçamento de amostragem, resolução do *raster* e distâncias da triangulação devem todos estar em unidade métrica.

As geometrias passam então por filtragem: feições vazias, inválidas ou sem altitude associada são removidas; geometrias compostas (**MultiLineString**) são decompostas em geometrias simples (**LineString**) para amostragem independente; e o campo de altitude é verificado quanto à validade numérica. Quando a equidistância E da base é conhecida, verifica-se a coerência das diferenças entre cotas:

$$|h_i - h_j| = mE, \quad m \in \mathbb{Z}. \quad (4.5)$$

Valores incompatíveis são marcados como possíveis inconsistências e inspecionados antes da interpolação, reduzindo o risco de propagar erros de atributo para o MDE final.

4.1.2.3 Amostragem em Nuvem de Pontos Tridimensionais

A transformação da representação vetorial em uma superfície *raster* parte da conversão das geometrias lineares em uma nuvem de pontos 3D. Seja uma curva parametrizada pelo comprimento acumulado:

$$\gamma(s) = (x(s), y(s)), \quad 0 \leq s \leq L, \quad (4.6)$$

em que L é o comprimento total da curva. Definido um espaçamento de amostragem Δs , os pontos extraídos de cada curva C_k , de cota h_k , são

$$s_j = j \Delta s, \quad p_{k,j} = (x(s_j), y(s_j), h_k), \quad (4.7)$$

de modo que cada ponto recebe as coordenadas planimétricas da curva e a altitude constante da isolinha. Ao final, obtém-se a nuvem de pontos

$$P = \{(x_i, y_i, z_i)\}_{i=1}^n. \quad (4.8)$$

Para compatibilidade com a grade *raster* a ser produzida com resolução r_{MDE} (Seção 4.1.2.5), adota-se o critério

$$\Delta s \leq r_{\text{MDE}}, \quad (4.9)$$

preservando o detalhe das curvas sem inflacionar desnecessariamente a nuvem de pontos.

4.1.2.4 Triangulação de Delaunay e Interpolação Baricêntrica

A partir da nuvem P , constrói-se uma rede triangular irregular (*Triangulated Irregular Network*, TIN) no plano XY segundo o critério de Delaunay, que evita triângulos excessivamente alongados e favorece uma malha geometricamente estável. Os pontos amostrados tornam-se vértices da malha; a superfície altimétrica é representada por planos definidos sobre cada triângulo.

Considerando um triângulo com vértices

$$A = (x_1, y_1, z_1), \quad B = (x_2, y_2, z_2), \quad C = (x_3, y_3, z_3), \quad (4.10)$$

a altitude de um ponto (x, y) no interior do triângulo é estimada por interpolação linear em coordenadas baricêntricas:

$$\hat{z}(x, y) = \lambda_1 z_1 + \lambda_2 z_2 + \lambda_3 z_3, \quad \lambda_1 + \lambda_2 + \lambda_3 = 1. \quad (4.11)$$

Os pesos λ_i correspondem às áreas relativas dos sub-triângulos. O método apresenta três propriedades favoráveis — formulação direta, baixo custo computacional e preservação exata das cotas dos pontos extraídos das curvas — que motivam a escolha da interpolação linear por TIN neste trabalho.

Limitação — efeito *terraço*. A interpolação linear direta sobre um TIN construído exclusivamente a partir de pontos amostrados de isolinhas é reconhecida na literatura de modelagem digital do terreno por produzir platôs e degraus artificiais entre curvas de nível consecutivas — o chamado efeito *terraço* (*terracing*) —, sobretudo em trechos de baixa declividade, onde a ausência de informação altimétrica interna às curvas (por exemplo, *breaklines* de cumeeiras e talwegues, ou pontos cotados adicionais) resulta em grandes regiões do TIN cobertas por triângulos quase paralelos à curva de nível mais próxima (BONIN; ROUSSEAU, 2005). O procedimento descrito nesta seção não incorpora nenhuma estratégia de mitigação desse efeito (inserção de *breaklines*, suavização pós-interpolação ou métodos alternativos como *splines* com tensão); os MDEs gerados a partir de curvas de nível vetoriais estão, portanto, sujeitos a essa distorção, cujo impacto sobre a comparação célula a célula com a topografia real ainda não foi quantificado (Seção 1.4).

4.1.2.5 Discretização em Grade, Exportação GeoTIFF e Normalização

A superfície contínua é discretizada em uma grade regular. Considerando os limites espaciais $x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$ e a resolução espacial r_{MDE} , os nós da grade são

$$x_m = x_{\min} + m r_{\text{MDE}}, \quad y_n = y_{\max} - n r_{\text{MDE}}, \quad (4.12)$$

e a matriz altimétrica resultante é $M_{\text{MDE}}[n, m] = \hat{f}(x_m, y_n)$, obtida por interpolação linear baricêntrica (equação (4.11)) sobre o TIN da Seção 4.1.2.4. Essa matriz é exportada como GeoTIFF via Rasterio, armazenando: valores altimétricos, sistema de referência de coordenadas, resolução espacial, transformação afim entre índices da matriz e coordenadas reais, e valor de *NoData* para regiões sem informação válida.

Como o caixão de areia possui altura física limitada, as cotas reais (que podem variar dezenas ou centenas de metros) não são transferidas diretamente — seja qual for a via de aquisição empregada (Seções 4.1.1 a 4.1.3): a superfície precisa ser **normalizada**, preservando a morfologia relativa. Sejam $Z_{\min}$ e $Z_{\max}$ as altitudes extremas do recorte; a altitude normalizada e sua conversão para a escala física da mesa são

$$Z_{\text{norm}}(x, y) = \frac{Z(x, y) - Z_{\min}}{Z_{\max} - Z_{\min}}, \quad Z_{\text{mesa}}(x, y) = h_{\max} Z_{\text{norm}}(x, y), \quad (4.13)$$

em que $h_{\max}$ é a altura máxima de areia modelável pelo sistema. Esse procedimento preserva a relação entre regiões altas e baixas do terreno, adaptando apenas sua amplitude vertical ao domínio físico do experimento, e produz a altura de referência $Z_{\text{mesa}}(x, y)$ utilizada pelo *AdaptadorMDE* na comparação célula a célula com a superfície real medida pelo Kinect, na sequência descrita nas Seções 4.4 e 4.6.

4.1.3 Aquisição a partir de Imagens SRTM, ASTER

Quando a área de interesse está fora da cobertura do BDGEx, ou quando o acesso autorizado a essa base não está disponível, o MDE de referência pode alternativamente

ser obtido a partir de modelos digitais de elevação de cobertura global, distribuídos gratuitamente em formato *raster* (GeoTIFF ou conversível via GDAL) e, portanto, consumidos pelo mesmo caminho direto descrito na Seção 4.1.1: leitura via rasterio, reprojeção quando necessário (equação (4.4)) e normalização (Seção 4.1.2.5).

SRTM. A *Shuttle Radar Topography Mission* (SRTM) mapeou, em fevereiro de 2000, cerca de 80% das massas continentais da Terra entre as latitudes 60°N e 60°S por interferometria de radar de abertura sintética a bordo do ônibus espacial *Endeavour*, em parceria entre NASA, NGA e as agências espaciais alemã e italiana (FARR et al., 2007). Os produtos globais mais recentes têm resolução de 1 arco-segundo (≈ 30 m) e acurácia vertical absoluta da ordem de 9 m (90% de confiança) (FARR et al., 2007).

ASTER GDEM. O *ASTER Global Digital Elevation Model* (GDEM), gerado por correlação estereoscópica de imagens ópticas do instrumento ASTER a bordo do satélite Terra (missão conjunta NASA/METI), oferece cobertura mais ampla que o SRTM — de 83°N a 83°S — na mesma resolução de 1 arco-segundo, porém com acurácia vertical tipicamente inferior à do SRTM (RMSE da ordem de 8,5 m contra 4,2 m do SRTM, em avaliação sobre o território dos EUA) e maior suscetibilidade a artefatos em áreas de cobertura de nuvens ou vegetação densa (GESCH et al., 2016).

Esta via de aquisição é compatível, em princípio, com a arquitetura do **AdaptadorMDE**, que já consome qualquer GeoTIFF válido independentemente da origem, mas não foi exercitada neste trabalho: os testes documentados no Capítulo 5 utilizam apenas dados sintéticos, e a validação com um MDE real proveniente de SRTM ou ASTER GDEM permanece como trabalho futuro (Seção 1.4).

4.2 Software de Projeção do MDE

4.2.1 Visão Geral — Arquitetura Monolítica Modularizada

A arquitetura adotada é **monolítica modularizada**: todos os módulos executam no mesmo processo Python, cada um encapsulando uma responsabilidade bem definida. Essa escolha garante:

- **Latência mínima** — comunicação por chamada de função em memória, essencial para o loop de tempo real;
- **Deploy simplificado** — ponto de entrada único (`main.py`);
- **Separação de responsabilidades** — cada módulo é testável e substituível de forma independente.

A Tabela 1 apresenta os módulos do sistema e suas responsabilidades.

Tabela 1 – Módulos do sistema AR Sandbox e suas responsabilidades

Módulo	Responsabilidade
<code>main.py</code>	Orquestrador: máquina de estados (INIT → IDLE → CALIBRACAO → AR_LOOP) + callback de mouse
<code>kinect_sensor.py</code>	Camada de hardware: <code>KinectSensor</code> com Open3D → free-nect → simulação interativa + grade persistente
<code>motor_caixao_areia.py</code>	Motor matemático: SVD, Gram-Schmidt, transformação 4×4, Tsai, discretização em grade, <code>fillPoly</code>
<code>mde_cartografia.py</code>	Adaptador de dados: <code>AdaptadorMDE</code> com GeoTIFF via rasterio, fallback Morro Gaussiano, heatmap
<code>test_motor_caixao.py</code>	Suíte TDD: 26 testes unitários automatizados

Fonte: Elaborado pelos autores.

4.2.2 Parâmetros Físicos do Sistema

Os parâmetros que governam o comportamento físico e numérico do sistema são concentrados no topo de `main.py` para facilitar a configuração. A Tabela 2 lista os parâmetros com seus valores padrão.

Tabela 2 – Parâmetros de configuração do sistema AR Sandbox

Parâmetro	Variável	Valor	Unidade
Largura da mesa	<code>LARGURA_MESA</code>	1,50	m
Comprimento da mesa	<code>COMPRIMENTO_MESA</code>	1,50	m
Altura máxima de areia	<code>ALTURA_MAX_AREIA</code>	0,30	m
Altura do Kinect	<code>ALTURA_KINECT</code>	2,50	m
Tolerância de cor	<code>TOLERANCIA_COR</code>	0,02	m
Células da grade (X)	<code>CELULAS_GRADE_X</code>	30	células
Células da grade (Y)	<code>CELULAS_GRADE_Y</code>	30	células
Raio da pá virtual	<code>RAIO_PA_VIRTUAL</code>	0,10	m
Intensidade da pá	<code>INTENSIDADE_PA_VIRTUAL</code>	0,008	m/evento

Fonte: Elaborado pelos autores.

4.2.3 Máquina de Estados

O módulo `main.py` implementa uma máquina de estados finita com quatro estados e transições controladas por `cv2.waitKey`:

INIT Inicializa `KinectSensor` (com fallback e grade persistente), carrega `AdaptadorMDE` (com fallback), cria janelas OpenCV, registra o callback de mouse via `cv2.setMouseCallback`.

IDLE Exibe a profundidade colorida do sensor; aguarda tecla **C** para calibrar.

CALIBRACAO No modo real: captura nuvem $\rightarrow$ SVD $\rightarrow$ Gram-Schmidt $\rightarrow$ Matriz 4×4 . No modo simulação: calibração automática ($T = I_4$).

AR_LOOP Loop contínuo: captura $\rightarrow$ transforma $\rightarrow$ discretiza em grade $\rightarrow$ compara MDE por célula $\rightarrow$ projeta vértices $\rightarrow$ `fillPoly`. O callback de mouse modifica a areia entre *frames*.

4.2.4 Padrões de Projeto

O sistema emprega quatro padrões de projeto clássicos, listados na Tabela 3.

Tabela 3 – Padrões de projeto utilizados no sistema AR Sandbox

Padrão	Localização	Propósito
Strategy + Fallback	<code>KinectSensor</code>	O construtor seleciona transparentemente a estratégia de captura (Open3D $\rightarrow$ freenect $\rightarrow$ simulação) sem alterar a interface pública
Adapter	<code>AdaptadorMDE</code>	Isola o formato de entrada (GeoTIFF, superfície sintética) da interface <code>obter_z_alvo(x, y)</code>
State Machine	<code>main.py</code>	Controla o fluxo do pipeline com estados bem definidos e transições determinísticas
Observer (Callback)	<code>main.py</code>	O callback de mouse observa eventos de interação e modifica o estado da areia de forma desacoplada

Fonte: Elaborado pelos autores.

4.2.5 Saídas Visuais

O sistema opera com duas janelas OpenCV simultâneas:

- **Projecao_Areia:** grade contínua de quadrados coloridos (vermelho/azul/verde) enviada ao projetor sobre a areia. Suporta tela cheia via `cv2.WINDOW_FULLSCREEN`. Aceita interação via mouse no modo simulação.
- **Gabarito_MDE:** *heatmap* 2D do MDE de referência com *colormap* — monitor auxiliar para o operador.

4.3 Motor Matemático — Pipeline Completo

O módulo `motor_caixao_areia.py` implementa os quatro pilares matemáticos que convertem uma nuvem de pontos bruta do Kinect em uma grade contínua de quadrados coloridos projetados sobre a areia. O pipeline opera em sequência, e cada passo é descrito com rigor formal nesta seção.

4.3.1 Passo 1 — Ajuste de Plano por SVD

Objetivo. Dada uma nuvem de N pontos $\{\mathbf{p}_i\}_{i=1}^N \subset \mathbb{R}^3$ capturados pelo Kinect sobre a superfície da mesa, encontrar o plano de melhor ajuste no sentido dos mínimos quadráticos.

Formulação. O plano é descrito por:

$$ax + by + cz + d = 0 \quad (4.14)$$

onde $\mathbf{n} = (a, b, c)^T$ é o vetor normal unitário e $d = -\mathbf{n} \cdot \bar{\mathbf{p}}$, com $\bar{\mathbf{p}}$ sendo o centroide da nuvem.

Algoritmo. O procedimento compreende três etapas:

1. **Centralização.** Calcula-se o centroide $\bar{\mathbf{p}} = \frac{1}{N} \sum_{i=1}^N \mathbf{p}_i$ e constrói-se a matriz centralizada:

$$M = \begin{bmatrix} (\mathbf{p}_1 - \bar{\mathbf{p}})^T \\ \vdots \\ (\mathbf{p}_N - \bar{\mathbf{p}})^T \end{bmatrix} \in \mathbb{R}^{N \times 3} \quad (4.15)$$

2. **Decomposição SVD.** Aplica-se $M = U\Sigma V^T$, com $\Sigma = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$, $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$.
3. **Extração da normal.** O último vetor-linha de V^T (associado a σ_3 , o menor valor singular) minimiza $\|M\mathbf{v}\|^2$ sujeito a $\|\mathbf{v}\| = 1$. Este vetor é a normal $\mathbf{n}$ do plano de melhor ajuste.

A convenção $n_z > 0$ garante que a normal aponta em direção ao Kinect (para cima).

Implementação: `ajustar_plano_svd()` em `motor_caixao_areia.py`.

4.3.2 Passo 2 — Base Ortonormal da Mesa (Gram-Schmidt)

Objetivo. Construir uma base ortonormal $\{X_{\text{mesa}}, Y_{\text{mesa}}, Z_{\text{mesa}}\}$ com $Z_{\text{mesa}} = \mathbf{n}$.

Método. A ortogonalização de Gram-Schmidt com produto vetorial procede em quatro etapas:

1. **Eixo Z:**

$$Z_{\text{mesa}} = \frac{\mathbf{n}}{\|\mathbf{n}\|} \quad (4.16)$$

2. **Vetor semente s:** escolhe-se $(1, 0, 0)^T$; se $|\mathbf{s} \cdot Z_{\text{mesa}}| \geq 0,9$, usa-se $(0, 1, 0)^T$ para evitar degeneração numérica.

3. Eixo X — Gram-Schmidt:

$$X_{\text{mesa}} = \frac{\mathbf{s} - (\mathbf{s} \cdot Z_{\text{mesa}}) Z_{\text{mesa}}}{\|\mathbf{s} - (\mathbf{s} \cdot Z_{\text{mesa}}) Z_{\text{mesa}}\|} \quad (4.17)$$

4. Eixo Y — Produto Vetorial:

$$Y_{\text{mesa}} = Z_{\text{mesa}} \times X_{\text{mesa}} \quad (4.18)$$

As condições de ortonormalidade são verificadas pela suíte de testes:

$$X_{\text{mesa}} \cdot Y_{\text{mesa}} = 0, \quad X_{\text{mesa}} \cdot Z_{\text{mesa}} = 0, \quad Y_{\text{mesa}} \cdot Z_{\text{mesa}} = 0, \quad \|X_{\text{mesa}}\| = \|Y_{\text{mesa}}\| = \|Z_{\text{mesa}}\| = 1 \quad (4.19)$$

Implementação: `gram_schmidt()` e `construir_base_mesa()` em `motor_caixao_areia.py`.

4.3.3 Passo 3 — Transformação Afim 4×4 (Kinect $\rightarrow$ Mesa)

Objetivo. Montar $T \in \mathbb{R}^{4 \times 4}$ tal que pontos sobre o plano da mesa tenham $z_{\text{mesa}} = 0$ e a origem coincida com o centroide $\bar{\mathbf{p}}$.

Construção. A matriz de rotação empilha os eixos da mesa como linhas:

$$R = \begin{bmatrix} X_{\text{mesa}}^T \\ Y_{\text{mesa}}^T \\ Z_{\text{mesa}}^T \end{bmatrix}, \quad \mathbf{t} = -R \bar{\mathbf{p}} \quad (4.20)$$

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (4.21)$$

Aplicação:

$$\mathbf{p}_{\text{mesa}} = T \cdot [\mathbf{p}_{\text{kinect}}^T, 1]^T \quad (4.22)$$

Propriedade fundamental: se $\mathbf{p}$ pertence ao plano, então $z_{\text{mesa}} = 0$.

No modo simulação, os pontos já estão em coordenadas da mesa, portanto $T = I_4$ (identidade).

Implementação: `montar_matriz_transformacao()` e `transformar_pontos()` em `motor_caixao_areia.py`.

4.3.4 Passo 4 — Projeção Pinhole/Tsai (3D → 2D)

Objetivo. Converter pontos 3D do referencial da mesa em pixels 2D do projetor.

O modelo de câmera *pinhole* com distorção de Tsai é implementado via `cv2.projectPoints`:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R_{\text{ext}} & \mathbf{t}_{\text{ext}} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (4.23)$$

onde a **matriz intrínseca** do projetor é:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (4.24)$$

e a **projeção em pixels** resulta em:

$$u = f_x \cdot \frac{X'}{Z'} + c_x, \quad v = f_y \cdot \frac{Y'}{Z'} + c_y \quad (4.25)$$

O modelo de distorção radial e tangencial de Tsai, com coeficientes $(k_1, k_2, p_1, p_2, k_3)$, é embutido no OpenCV e corrige aberrações ópticas do projetor (TSAI, 1987; OpenCV Team, 2024).

Implementação: `projetar_pontos_tsai()` e `calibrar_projetor()` em `motor_caixao_are`

Lacuna em aberto — calibração extrínseca mesa→projetor. Diferentemente da calibração Kinect→mesa, obtida pelo procedimento SVD + Gram-Schmidt das Seções 4.3.1 e 4.3.2, este texto não descreve o método pelo qual R_{ext} e $\mathbf{t}_{\text{ext}}$ da Equação (4.23) — os extrínsecos que relacionam o referencial da mesa ao referencial do projetor — são efetivamente obtidos, apesar de existir a função `calibrar_projetor()` citada acima. Fisicamente, o Kinect e o projetor ocupam posições distintas sobre a mesa, de modo que não é correto reaproveitar diretamente a matriz extrínseca obtida na calibração do sensor. Por essa razão, nenhuma alegação de projeção geometricamente correta é feita neste trabalho (Seção 4.4); essa lacuna deve ser sanada documentando-se o método de calibração projetor–mesa de fato empregado (por exemplo, por correspondência de pontos de controle projetados e capturados pelo próprio Kinect, ou por um padrão de calibração conhecido) antes da validação com hardware real (Seção 1.4).

4.4 Renderização por Malha Discretizada

O pipeline de renderização substitui a abordagem de projeção de pontos isolados por uma **discretização em malha regular** que garante cobertura visual contínua, filtragem de ruído do sensor e projeção geométrica eficiente.

4.4.1 Definição da Grade

O plano da mesa $[0, L_x] \times [0, L_y]$, com $L_x = L_y = 1,50$ m, é particionado em uma grade uniforme de $N_x \times N_y$ células retangulares (padrão: $N_x = N_y = 30$). Cada célula $C_{i,j}$ ocupa o domínio:

$$C_{i,j} = [j \cdot \Delta x, (j+1)\Delta x] \times [i \cdot \Delta y, (i+1)\Delta y] \quad (4.26)$$

onde os tamanhos de célula são:

$$\Delta x = \frac{L_x}{N_x} = \frac{1,50}{30} = 0,05 \text{ m}, \quad \Delta y = \frac{L_y}{N_y} = \frac{1,50}{30} = 0,05 \text{ m} \quad (4.27)$$

A grade possui $(N_y + 1) \times (N_x + 1) = 31 \times 31 = 961$ vértices.

Justificativa da resolução. O valor $N_x = N_y = 30$ (células de 5×5 cm) foi adotado como compromisso qualitativo entre três fatores, sem que uma análise quantitativa de sensibilidade tenha sido conduzida: (i) **granularidade útil ao operador** — células menores que a pá de trabalho típica (ferramentas manuais de jardinagem, da ordem de centímetros) não trariam ganho de precisão perceptível na tarefa de modelagem manual da areia; (ii) **compatibilidade com o ruído do sensor** — células muito pequenas reduziram o número médio de pontos Kinect por célula ($|\mathcal{P}_{i,j}|$ na Equação (4.28)), prejudicando a filtragem por média discutida na Seção 4.4.3 a seguir; e (iii) **desempenho de renderização** — o custo de `fillPoly` cresce com o número de células. Não foram testadas alternativas (por exemplo, grades de 15×15 ou 60×60 células) nem medido o efeito da resolução sobre a acurácia percebida da comparação; essa análise é apontada como trabalho futuro (Seção 1.4).

4.4.2 Agregação Espacial — Média de Altura por Célula

Para cada célula $C_{i,j}$, define-se:

$$\mathcal{P}_{i,j} = \left\{ \mathbf{p}_k = (x_k, y_k, z_k) \in \text{nuvem} \mid (x_k, y_k) \in C_{i,j} \right\} \quad (4.28)$$

A altura representativa é a média aritmética:

$$\bar{Z}_{i,j}^{\text{real}} = \frac{1}{|\mathcal{P}_{i,j}|} \sum_{\mathbf{p}_k \in \mathcal{P}_{i,j}} z_k \quad (4.29)$$

Justificativa. O sensor efetivamente utilizado (Kinect v2) apresenta ruído que cresce com a distância de operação, com comportamento distinto do Kinect v1 (Seção 2). Na ausência de uma caracterização própria do ruído do v2 na distância de trabalho de 2,5 m, adota-se, como estimativa conservadora para o dimensionamento do filtro por agregação, o valor de referência $\sigma \approx 3$ mm reportado para o Kinect v1 a 2–3 m (KHOSHELHAM; ELBERINK, 2012); essa adoção é uma aproximação, não uma medição do sensor real, e é tratada como limitação do escopo (Seção 1.4). Sob essa aproximação, ao agregar n pontos por célula, a média reduz o desvio padrão do estimador para $\sigma_{\bar{z}} = \sigma/\sqrt{n}$, proporcionando filtragem significativa sem custo computacional adicional — a magnitude real desse ganho para o Kinect v2 a 2,5 m permanece, contudo, por verificar.

A implementação vetorizada usa `np.add.at` para acumulação sem *loops* Python:

```
1 col = np.clip((x / tam_celula_x).astype(np.int32), 0, n_celulas_x - 1)
2 lin = np.clip((y / tam_celula_y).astype(np.int32), 0, n_celulas_y - 1)
3 np.add.at(soma_z, (lin, col), z)
4 np.add.at(contagens, (lin, col), 1)
5 media_z = np.where(contagens > 0, soma_z / contagens, np.nan)
```

Listing 4.1 – Implementação vetorizada do *binning* espacial

Células sem pontos ($|\mathcal{P}_{i,j}| = 0$) são marcadas como NaN e não são renderizadas.

4.4.3 Regra de Negócio — Coloração por Diferença de Altitude

O centro geométrico de cada célula $C_{i,j}$ é:

$$\mathbf{c}_{i,j} = \left((j + \frac{1}{2})\Delta x, (i + \frac{1}{2})\Delta y \right) \quad (4.30)$$

A altura alvo do MDE é consultada nesse centro: $Z_{i,j}^{\text{MDE}} = \text{obter_z_alvo}(\mathbf{c}_{i,j})$.

A cor atribuída segue a regra parametrizada pela tolerância $\tau = 0,02$ m:

$$\text{cor}(i, j) = \begin{cases} \text{Vermelho} (0, 0, 255)_{\text{BGR}} & \text{se } \bar{Z}_{i,j}^{\text{real}} > Z_{i,j}^{\text{MDE}} + \tau \quad (\text{cavar}) \\ \text{Azul} (255, 0, 0)_{\text{BGR}} & \text{se } \bar{Z}_{i,j}^{\text{real}} < Z_{i,j}^{\text{MDE}} - \tau \quad (\text{preencher}) \\ \text{Verde} (0, 255, 0)_{\text{BGR}} & \text{caso contrário} \quad (\text{OK}) \end{cases} \quad (4.31)$$

Nota: As cores são representadas em convenção BGR (padrão do OpenCV). As comparações são estritas ($>$ e $<$), portanto no limite exato a classificação é verde.

Limitação — descontinuidade binária. A regra da Equação (4.31) classifica cada célula em apenas três categorias discretas a partir de comparações estritas com a tolerância τ , o que introduz uma descontinuidade artificial: duas células com diferenças de $-0,021$ m e $-0,019$ m em relação ao MDE recebem cores distintas (azul e verde, respectivamente) apesar de a diferença física entre elas ser desprezível, enquanto duas células com diferenças de $-0,019$ m e $0,019$ m — fisicamente também próximas — permanecem ambas verdes. O sistema não oferece, portanto, indicação visual gradual de *quão distante* uma célula está do alvo dentro de cada faixa, apenas a categoria à qual pertence. Uma extensão natural seria substituir a classificação discreta por um mapeamento contínuo (por exemplo, intensidade de cor proporcional à magnitude do desvio, saturando além de um limiar), preservando a leitura binária cavar/preencher/OK como caso particular. Essa limitação é reconhecida mas não mitigada na versão atual do sistema.

4.4.4 Projeção em Lote e Rasterização

A grade de $(N_y + 1) \times (N_x + 1)$ vértices é projetada em uma **única chamada** a `cv2.projectPoints`, com $Z = 0$ (plano da mesa):

$$\text{vértices_2d} = \text{Tsai}\left(\left\{\left(\frac{jL_x}{N_x}, \frac{iL_y}{N_y}, 0\right)\right\}_{i=0,\dots,N_y}^{j=0,\dots,N_x}\right) \quad (4.32)$$

Cada célula $C_{i,j}$ é desenhada como quadrilátero preenchido pelos quatro vértices adjacentes:

$$\text{quad}_{i,j} = [V_{i,j}, V_{i,j+1}, V_{i+1,j+1}, V_{i+1,j}] \quad (4.33)$$

A rasterização via `cv2.fillPoly(imagem, [quad], cor)` garante **cobertura contínua** (sem buracos) e **eficiência** (projeção em $O(1)$ chamadas ao OpenCV). A correção geométrica da projeção resultante depende da calibração extrínseca mesa→projektor, cujo método não está documentado neste trabalho (Seção 4.3.4); nenhuma alegação de correção geométrica é feita até que essa lacuna seja sanada e validada com hardware real (Seção 1.4).

4.5 Emulador Interativo de Areia

Na ausência de hardware físico, o sistema oferece um emulador interativo que permite ao operador modificar a topografia virtual em tempo real, demonstrando o pipeline completo de forma tangível para a banca avaliadora.

4.5.1 Estado Persistente da Areia

A classe `KinectSensor`, em modo simulação, inicializa e mantém uma **matriz de alturas persistente** $\mathbf{H} \in \mathbb{R}^{R \times R}$ (com $R = 50$):

$$H_{i,j}^{(0)} = \frac{h_{\max}}{2} = 0,15 \text{ m} \quad \forall i, j \quad (4.34)$$

Os eixos da grade mapeiam linearmente para as coordenadas físicas da mesa:

$$x_j = \frac{j \cdot L_x}{R-1}, \quad y_i = \frac{i \cdot L_y}{R-1}, \quad i, j \in \{0, \dots, R-1\} \quad (4.35)$$

4.5.2 Modelo Matemático da Pá Virtual

Ao receber um evento de mouse na posição física $\mathbf{m} = (x_m, y_m)$, o sistema aplica a atualização:

$$H_{i,j}^{(t+1)} = \text{clip} \left(H_{i,j}^{(t)} \pm \alpha \cdot \exp \left(-\frac{(x_j - x_m)^2 + (y_i - y_m)^2}{2\sigma^2} \right), 0, h_{\max} \right) \quad (4.36)$$

onde os parâmetros estão descritos na Tabela 4.

Tabela 4 – Parâmetros do modelo da pá virtual

Símbolo	Significado	Valor padrão	Unidade
α	Intensidade do deslocamento por evento	0,008	m
σ	Desvio padrão do perfil Gaussiano	0,05 ($= r/2$)	m
r	Raio de ação da pá virtual	0,10	m
+	Operador ao preencher (botão direito)	—	—
—	Operador ao cavar (botão esquerdo)	—	—

Fonte: Elaborado pelos autores.

Justificativa do perfil Gaussiano. A distribuição Gaussiana 2D produz deformação suave, isenta de artefatos de escavações retangulares. Com $\sigma = r/2$, aproximadamente 86,5% da energia concentra-se dentro do raio r :

$$\frac{\int_0^r 2\pi\rho e^{-\rho^2/2\sigma^2} d\rho}{\int_0^\infty 2\pi\rho e^{-\rho^2/2\sigma^2} d\rho} = 1 - e^{-r^2/2\sigma^2} = 1 - e^{-2} \approx 0,865 \quad (4.37)$$

4.5.3 Mapeamento Pixel $\rightarrow$ Coordenada Física

O callback registrado via `cv2.setMouseCallback` converte a posição do cursor (u, v) em pixels para coordenadas físicas (x, y) da mesa:

$$x_{\text{mesa}} = \frac{u}{W_{\text{janela}}} \cdot L_x, \quad y_{\text{mesa}} = \frac{v}{H_{\text{janela}}} \cdot L_y \quad (4.38)$$

onde W_{janela} e H_{janela} são as dimensões atuais da janela (obtidas via `cv2.getWindowImageRect()`), garantindo funcionamento correto mesmo com redimensionamento.

4.6 Mecanismo de Resiliência — Fallback em Cascata

O sistema foi projetado com o requisito inviolável de nunca falhar durante a apresentação à banca, independentemente do hardware ou arquivos disponíveis.

4.6.1 Sensor — KinectSensor

O construtor de `KinectSensor` tenta cada estratégia de captura em ordem de preferência, recuando para a seguinte sempre que a anterior não estiver disponível:

1. **pykinect2** (Kinect Xbox One S / v2) — estratégia principal deste trabalho, via *Kinect for Windows SDK 2.0*; a adaptação de hardware e as correções de software que a viabilizam são descritas na Seção 4.7.
2. **Open3D** (Azure Kinect / RealSense) — se o Kinect v2 não estiver disponível e um sensor compatível com Open3D estiver presente, usa.
3. **freenect** (Kinect v1) — se as anteriores falharem e a `libfreenect` estiver disponível, usa.
4. **Simulação Interativa** — todas falharam: grade persistente **H** inicializada em 0,15 m; método `modificar_areia()` disponível via mouse.

4.6.2 MDE — AdaptadorMDE

O construtor de `AdaptadorMDE` tenta:

1. Resolução do caminho via `pathlib`.
2. Leitura do GeoTIFF com rasterio, normalização para [0, 0,30] m, interpolador bilinear via `scipy.interpolate.RegularGridInterpolator`.
3. Arquivo inexistente / rasterio ausente / erro: gera o **Morro Gaussiano** sintético:

$$Z_{\text{MDE}}(x, y) = 0,3 \cdot \exp\left(-\frac{(x - 0,75)^2 + (y - 0,75)^2}{0,1}\right) \quad (4.39)$$

4.6.3 Estado Visual Inicial da Simulação

Quando Kinect e GeoTIFF estão ambos ausentes, a combinação dos dois *mocks* define o estado inicial mostrado na Tabela 5.

Tabela 5 – Estado visual inicial no modo simulação completo

Região	$\bar{Z}^{\text{real}}$	Z^{MDE}	Diferença	Cor
Bordas	0,15 m	$\approx 0,00$ m	$+0,15 > +0,02$	Vermelho
Anel int.	0,15 m	$\approx 0,15$ m	≈ 0	Verde
Centro	0,15 m	$\approx 0,30$ m	$-0,15 < -0,02$	Azul

Fonte: Elaborado pelos autores.

4.7 Adaptação e Integração do Sensor Kinect Xbox One S

Conforme antecipado na Seção 4.6, a estratégia principal de captura emprega um **Kinect Xbox One S (v2)**, adaptado para conexão direta ao PC mediante modificação de hardware. Como o barramento de dados e a alimentação desse sensor dependem de um periférico proprietário, esta seção detalha a adaptação de hardware e as correções de software necessárias para integrá-lo ao *pipeline* descrito nas seções anteriores. O *Kinect Adapter* oficial da Microsoft, periférico necessário para conectar o sensor a um computador, encontra-se descontinuado e indisponível no mercado nacional a preço acessível, inviabilizando sua aquisição dentro do orçamento do projeto. Dessa forma, a solução adotada consistiu em realizar a adaptação diretamente no hardware do sensor, substituindo o conector proprietário por componentes de baixo custo e uma fonte de alimentação externa.

4.7.1 Adaptação de Hardware

O procedimento consistiu em abrir o sensor, remover o conector proprietário e soldar diretamente na placa os pontos de alimentação (VCC 12 V e GND). Para realizar esse procedimento, foram utilizados os materiais listados na Tabela 6.

Tabela 6 – Materiais utilizados na adaptação do Kinect Xbox One S

Componente	Especificação	Custo
Cabo USB AB 3.0	1,5 metros	R\$ 39,00
Conector P4 fêmea	Alimentação 12 V	—
Fonte de bancada	Protec S-180-12 · 12 V / 15 A	—

Fonte: Elaborado pelos autores.

A interface de dados é estabelecida pelo cabo USB 3.0 AB (porta USB 3.0 obrigatória no PC), e a alimentação é fornecida por uma fonte de bancada de 12 V via conector P4 fêmea soldado nos pontos de VCC e GND da placa. Medições realizadas em bancada

confirmaram consumo entre 1,5 A e 2,0 A em operação normal, o que indica que a fonte de 15 A é superdimensionada por segurança: qualquer fonte estabilizada de 12 V / 3 A atende aos requisitos do sensor. O funcionamento correto após a modificação foi confirmado pela ativação do emissor infravermelho (IR) do sensor, visível como luz roxa característica, e pela transmissão bem-sucedida da imagem RGB ao notebook durante os testes de bancada.

4.7.2 Compatibilidade de Software — pykinect2 e Python 3.12

A biblioteca `pykinect2`, que faz a ponte entre o Kinect v2 e o *Kinect for Windows SDK 2.0* da Microsoft, foi originalmente desenvolvida para Python 2 e Python 3.6. Sua adoção sob Python 3.12 com NumPy 1.26.4 exigiu cinco correções manuais, motivadas pela remoção de APIs obsoletas (`numpy.distutils`, `time.clock()` e `dtype=numpy.object`) e por incompatibilidades no tamanho de uma estrutura COM e na versão do `comtypes`. O detalhamento de cada correção, com os respectivos pontos de alteração no código, é apresentado no Apêndice B. As versões fixadas para garantir compatibilidade são `numpy==1.26.4` e `comtypes==1.3.1`.

4.7.3 Script de Diagnóstico e Validação

Para verificar toda a cadeia de hardware e software antes de executar o projeto principal, foi desenvolvido o script `diagnostico_kinect.py`, que executa seis verificações sequenciais:

1. Arquitetura do Python (64-bit obrigatório);
2. Presença da variável de ambiente `KINECTSDK20_DIR` (SDK instalado);
3. Importação do pacote `pykinect2`;
4. Importação do `PyKinectRuntime` (valida todas as correções descritas no Apêndice B);
5. Abertura do sensor via `PyKinectRuntime`;
6. Recepção do primeiro *frame* de profundidade em até 5 segundos.

O resultado obtido nos testes de bancada confirmou o funcionamento completo do sensor, entregando os pixels capturados por *frame*, com distância média ao objeto mais próximo no campo de visão.

5 RESULTADOS

Conforme delimitado na Seção 1.4, os resultados apresentados neste capítulo cobrem exclusivamente a **correção matemática do motor de cálculo**, verificada por testes unitários com valores sintéticos. Não são apresentados aqui resultados de validação end-to-end com hardware real, medições de desempenho (taxa de quadros, latência) ou de acurácia de reprojeção geométrica, os quais permanecem como trabalho futuro (Seção 6.2).

5.1 Estratégia TDD

A estabilidade do motor matemático foi assegurada com *Test-Driven Development* (TDD), resultando em **26 testes unitários** em `test_motor_caixao.py`, dos quais 25 são executados em qualquer ambiente e 1 é ignorado (*skipped*) automaticamente quando o Open3D não está instalado. Os testes usam valores *hardcoded* com comentários que explicitam a conta passo a passo, permitindo verificação pela banca sem execução do código.

Para executar a suíte:

```
1 python -m unittest test_motor_caixao -v
2 # Resultado esperado: 25 passed, 1 skipped (Open3D ausente)
```

Listing 5.1 – Execução da suíte de testes

5.2 Cobertura por Componente

A Tabela 7 apresenta a cobertura de testes por componente.

5.3 Exemplos de Transparência Matemática

Plano horizontal em $z = 5$:

```
1 # Pontos: (0,0,5), (1,0,5), (0,1,5), (1,1,5), (2,3,5)
2 # Normal esperada: (0, 0, 1) - plano horizontal
3 # d esperado: -5
4 # Verificacao: n . p + d = (0,0,1).(0,0,5) + (-5) = 5 - 5 = 0 OK
5 pontos = np.array([[0,0,5],[1,0,5],[0,1,5],[1,1,5],[2,3,5]])
6 n, d = ajustar_plano_svd(pontos)
7 np.testing.assert_allclose(np.abs(n), [0,0,1], atol=1e-6)
8 np.testing.assert_allclose(np.abs(d), 5.0, atol=1e-6)
```

Listing 5.2 – Teste unitário: ajuste de plano horizontal

Tabela 7 – Cobertura da suíte de testes unitários

Classe de Teste	Qtd	Componente Verificado
TestAjustePlano	4	SVD: normal unitária, plano horizontal $z = 5$, plano inclinado, exceção $N < 3$
TestGramSchmidt	2	Ortogonalidade, exceção para vetores paralelos
TestConstruirBase	3	Ortonormalidade mútua, $Z_{\text{mesa}} = \mathbf{n}$, planos inclinados
TestMatrizTransformacao	3	$T = I$ para base canônica, translação anula origem, $z_{\text{mesa}} = 0$
TestDeteccaoTabuleiro	2	Imagem sem tabuleiro $\rightarrow$ False , tabuleiro sintético 7×5
TestProjecaoTsai	3	Ponto principal, deslocamento em x , múltiplos pontos
TestLeituraRGBD	1	Importação condicional Open3D (skip gracioso se ausente)
TestMDEColoracao	6	Vermelho/Azul/Verde, limites exatos, mock rampa linear
TestPipeline	2	Integração plano $z = 0$ e ponto acima $z = 10$
Total	26	—

Fonte: Elaborado pelos autores.

Projeção *pinhole*:

```

1 # Ponto (0,0,0), tvec = (0,0,1), fx=320, cx=320, fy=240, cy=240
2 # u = 320 * (0/1) + 320 = 320
3 # v = 240 * (0/1) + 240 = 240
4 # Pixel esperado: (320, 240) - centro da imagem
5 pixel = projetar_pontos_tsai(np.array([[0,0,0]]), K, dist, R_id,
6                             [0,0,1])
6 np.testing.assert_allclose(pixel[0], [320, 240], atol=0.5)

```

Listing 5.3 – Teste unitário: projeção *pinhole* no ponto principal

6 CONCLUSÃO

6.1 Conclusões

Este trabalho apresentou o desenvolvimento completo de um sistema AR Sandbox capaz de operar em tempo real sobre hardware físico (sensor Kinect + projetor) ou em modo de simulação interativa completa, sem dependência de hardware externo.

Os principais resultados alcançados foram:

1. **Pipeline matemático robusto:** SVD para ajuste de plano, Gram-Schmidt para base ortonormal, transformação afim 4×4 e projeção Tsai, todos implementados em Python puro com NumPy.
2. **Renderização contínua:** a discretização em malha de 30×30 células com rasterização por `fillPoly` elimina buracos e ruído pontual da projeção.
3. **Resiliência total:** o mecanismo de *fallback* em cascata garante que o sistema nunca trave, operando degradadamente mas de forma funcional mesmo sem hardware.
4. **Emulador interativo:** a pá virtual com perfil Gaussiano permite demonstrar o pipeline completo para a banca avaliadora sem a necessidade da caixa de areia física.
5. **Qualidade de software:** 26 testes unitários com cobertura completa do motor matemático, executáveis sem instalação de hardware.

6.2 Trabalhos Futuros

Como discutido na Seção 1.4, a validação de ponta a ponta do sistema ainda não foi realizada; as extensões a seguir cobrem tanto essa validação pendente quanto funcionalidades adicionais:

- **Validação com hardware real:** validar a captura de profundidade sobre a superfície de areia real ($1,5\text{ m} \times 1,5\text{ m}$) com o sensor posicionado a 2,5 m de altura e realizar a calibração geométrica (SVD + Gram-Schmidt) no posicionamento definitivo;
- **Ajuste de parâmetros:** refinar os parâmetros físicos do sistema (`ALTURA_KINECT`, `CELULAS_GRADE_X/Y`, `TOLERANCIA_COR`) com base nas medições reais sobre a mesa;
- **Validação cartográfica:** validar o pipeline completo com MDE real proveniente do BDGEx e corrigir eventuais erros de mapeamento geométrico entre o referencial do sensor e o referencial da mesa;

- **Simulação hidrológica:** integrar um *solver* de equações de águas rasas para simular escoamento sobre o terreno modelado.
- **Calibração automática:** implementar calibração automática do projetor por detecção de padrões estruturados projetados na superfície.
- **Múltiplos MDEs:** suporte a troca de MDE em tempo real via interface gráfica.
- **Exportação GeoTIFF:** exportar o terreno modelado como GeoTIFF georreferenciado para uso em SIG.
- **Interface web:** disponibilizar uma versão web do emulador para uso educacional remoto.

REFERÊNCIAS

BONIN, O.; ROUSSEAU, F. Digital terrain model computation from contour lines: How to derive quality information from artifact analysis. *GeoInformatica*, v. 9, n. 3, p. 253–268, 2005.

FARR, T. G.; ROSEN, P. A.; CARO, E.; CRIPPEN, R.; DUREN, R.; HENSLEY, S.; KOBRICK, M.; PALLER, M.; RODRIGUEZ, E.; ROTH, L.; SEAL, D.; SHAFFER, S.; SHIMADA, J.; UMLAND, J.; WERNER, M.; OSKIN, M.; BURBANK, D.; ALSDORF, D. The shuttle radar topography mission. *Reviews of Geophysics*, v. 45, n. 2, p. RG2004, 2007.

GESCH, D.; OIMOEN, M.; DANIELSON, J.; MEYER, D. Validation of the ASTER global digital elevation model version 3 over the conterminous united states. In: *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*. [S.l.: s.n.], 2016. XLI-B4, p. 143–148.

GILLIES, S. et al. *Rasterio: Geospatial Raster I/O for Python Programmers*. 2024. Acesso em: mar. 2026. Disponível em: <<https://rasterio.readthedocs.io/>>.

GOLUB, G. H.; LOAN, C. F. V. *Matrix Computations*. 4. ed. [S.l.]: Johns Hopkins University Press, 2013. ISBN 978-1421407944.

HARTLEY, R.; ZISSERMAN, A. *Multiple View Geometry in Computer Vision*. 2. ed. [S.l.]: Cambridge University Press, 2003. ISBN 978-0521540513.

KHOSHELHAM, K.; ELBERINK, S. O. Accuracy and resolution of kinect depth data for indoor mapping applications. *Sensors*, v. 12, n. 2, p. 1437–1454, 2012.

KREYLOS, O. *SARndbox: Augmented Reality Sandbox*. 2016. Acesso em: mar. 2026. Disponível em: <<https://arsandbox.ucdavis.edu/>>.

KREYLOS, O.; CAVES, W. K. Augmented reality sandbox. *UC Davis W.M. Keck Center for Active Visualization in the Earth Sciences (KeckCAVES)*, 2013. Disponível em: <<https://github.com/KeckCAVES/SARndbox>>.

LI, Z.; ZHU, Q.; GOLD, C. *Digital Terrain Modeling: Principles and Methodology*. Boca Raton: CRC Press, 2005. ISBN 0-415-32462-9.

LÖHR, J. *AR-Sandbox*. 2019. Acesso em: mai. 2026. Disponível em: <<https://www.julianloehr.de/educational-work/ar-sandbox/>>.

OpenCV Team. *OpenCV: Open Source Computer Vision Library*. 2024. Acesso em: mar. 2026. Disponível em: <<https://docs.opencv.org/4.x/>>.

OpenKinect Community. *libfreenect: Drivers and Libraries for the Xbox Kinect Device on Windows, Linux, and OS X*. 2023. Acesso em: mar. 2026. Disponível em: <<https://github.com/OpenKinect/libfreenect>>.

- REED, S.; KREYLOS, O.; HSI, S.; KELLOGG, L.; SCHLADOW, G.; YIKILMAZ, M. B.; SEGALE, H.; SILVERMAN, J.; YALOWITZ, S.; SATO, E. *Shaping Watersheds Exhibit: An Interactive, Augmented Reality Sandbox for Advancing Earth Science Education*. 2014. American Geophysical Union (AGU) Fall Meeting, Abstract ED34A-01.
- TÖLGYESSY, M.; DEKAN, M.; CHOVANEC, L.; HUBINSKÝ, P. Evaluation of the azure Kinect and its comparison to Kinect V1 and Kinect V2. *Sensors*, v. 21, n. 2, p. 413, 2021.
- TSAI, R. Y. A versatile camera calibration technique for high-accuracy 3d machine vision metrology using off-the-shelf tv cameras and lenses. *IEEE Journal on Robotics and Automation*, v. 3, n. 4, p. 323–344, 1987.
- WASENMÜLLER, O.; STRICKER, D. Comparison of kinect V1 and V2 depth images in terms of accuracy and precision. In: *Computer Vision – ACCV 2016 Workshops*. [S.l.]: Springer, 2017. (Lecture Notes in Computer Science, v. 10117), p. 34–45.
- WOODS, T. L.; REED, S.; HSI, S.; WOODS, J. A.; WOODS, M. R. Pilot study using the augmented reality sandbox to teach topographic maps and surficial processes in introductory geology labs. *Journal of Geoscience Education*, v. 64, n. 3, p. 199–214, 2016.
- YAMAZAKI, D.; IKESHIMA, D.; TAWATARI, R.; YAMAGUCHI, T.; O’LOUGHLIN, F.; NEAL, J. C.; SAMPSON, C. C.; KANAE, S.; BATES, P. D. A high-accuracy map of global terrain elevations. *Geophysical Research Letters*, v. 44, n. 11, p. 5844–5853, 2017.
- ZHOU, Q.-Y.; PARK, J.; KOLTUN, V. Open3d: A modern library for 3d data processing. *arXiv preprint arXiv:1801.09847*, 2018. Disponível em: <<https://arxiv.org/abs/1801.09847>>.

APÊNDICE A – CRONOGRAMA DE EXECUÇÃO DO PROJETO

Tabela 8 – Cronograma previsto para o desenvolvimento do projeto.

Atividade	Fev–Mar	Abr–Mai	Jun–Jul	Ago–Set	Out
Revisão bibliográfica	X	X			
Aquisição e processamento das curvas de nível (BDGEx)	X	X			
Geração do MDE de referência (TIN + GeoTIFF)		X	X		
Implementação do motor matemático (SVD, Gram-Schmidt, Tsai)			X	X	
Renderização por malha discretizada e emulador interativo			X	X	
Calibração e integração com hardware físico (Kinect + projetor)				X	
Testes unitários e validação (TDD)			X	X	X
Redação do relatório final e preparação da defesa				X	X

Fonte: Elaborado pelos autores.

APÊNDICE B – CORREÇÕES DA BIBLIOTECA PYKINECT2 PARA PYTHON 3.12

A biblioteca `pykinect2`, utilizada para comunicação com o Kinect v2 via SDK da Microsoft, foi originalmente desenvolvida para Python 2 e Python 3.6. Conforme mencionado na Seção 4.7, sua operação com Python 3.12 e NumPy 1.26.4 exigiu cinco correções manuais, detalhadas a seguir:

1. **`numpy.distutils` removido no NumPy 2.0+** — substituído por um *mock* mínimo com `platform_bits = 64`, via bloco `try/except ImportError` em `PyKinectV2.py` (linha 24);
2. **`sizeof(tagSTATSTG)` incorreto no Python 3.12** — o tamanho da estrutura COM `tagSTATSTG` passou de 72 para 80 bytes; a asserção na linha 2216 de `PyKinectV2.py` foi atualizada para `assert sizeof(tagSTATSTG) == 80`;
3. **`_check_version` incompatível com `comtypes 1.3.1`** — a chamada `_check_version()` na linha 2874 de `PyKinectV2.py` foi comentada, pois a versão 1.4.x do `comtypes` quebra a interface COM do sensor;
4. **`time.clock()` removido no Python 3.8+** — todas as ocorrências em `PyKinectRuntime.py` foram substituídas por `time.perf_counter()`;
5. **`dtype=numpy.object` removido no NumPy 1.24+** — todas as ocorrências em `PyKinectRuntime.py` foram substituídas por `dtype=object`.

As versões fixadas para garantir compatibilidade são `numpy==1.26.4` e `comtypes==1.3.1`.